Project Proposal: Steam Game Recommendation System

Course: IT4868 – Web Mining (SoICT, HUST)
Lecturer: Dr. Quang Huy Duong
Topic: Recommendation Systems
Approach: Academic (problem definition, proposed methods, experiments on benchmarks + code, conclusion)

Team members:

No.	Full name	Student ID
1	Trần Quang Huy	202416706
2	Phạm Châu Long	202416714
3	Vũ Hồng Nam	202416728
4	Phạm Minh Hiếu	202416693

1. Problem Definition

1.1. Background

Steam is the world's largest PC game distribution platform, with more than 100,000 titles and tens of millions of active users. The large number of games released every year makes it hard for users to find games that suit them on their own. A recommender system is a core component for personalizing the experience and improving game discovery, especially for lesser-known indie games.

Unlike movie data (e.g., MovieLens) with explicit 1–5 star ratings, Steam data is mostly **implicit feedback**: game ownership, playtime, and binary reviews (Recommended / Not Recommended). This poses its own modeling challenges.

1.2. Problem Statement

Top-K Recommendation from implicit feedback:

- **Input:**
  - A set of users U and a set of games I.
  - The interaction history of each user u: a sequence of games $(i_1, i_2, \ldots, i_n)$ with timestamps, playtime, and review labels (recommended or not).
  - Game metadata: tags, genres, description, price, release date.
- **Output:** A ranked list of K games (e.g., $K = 10, 20$) that user u has not interacted with and is most likely to play/like next.

Formally: learn a scoring function $\hat{y}_{ui} = f(u, i \mid \Theta)$ and return $\text{Top-}K_{i \in I \setminus I_u}\ \hat{y}_{ui}$.

1.3. Challenges

1. **Implicit feedback & missing negatives:** A game the user has not interacted with is not necessarily disliked — the user may simply not know about it.
2. **Extreme sparsity:** Each user interacts with only a tiny fraction of tens of thousands of games (density is typically < 0.1%).
3. **Popularity bias / long tail:** A small number of AAA games account for most interactions; models tend to recommend only popular games.
4. **Cold start:** Newly released games have few or no interactions.
5. **Sequential dynamics:** User preferences change over time; recently played games reflect current interests better.
6. **Large scale:** Tens of millions of interactions — processing must be efficient within limited resources (Google Colab).

1.4. Research Questions

- **RQ1:** How much do graph-based (LightGCN) and sequential (SASRec) deep models improve over classical CF baselines on Steam data?
- **RQ2:** Does injecting game content (tags, descriptions) into item embeddings improve recommendation for cold-start and long-tail games?
- **RQ3:** Does weighting interactions by playtime and review sentiment produce better rankings than treating all interactions equally?

2. Proposed Methods

We will implement and compare three groups of methods, from classical approaches to Deep Learning, and propose a hybrid improvement.

2.1. Baselines

Model	Short description	Motivation
MostPop	Recommend the most popular games the user does not own	Minimal baseline; measures popularity bias
ItemKNN	Collaborative filtering based on cosine similarity between items	Classic CF method, easy to interpret
BPR-MF	Matrix Factorization optimized with the Bayesian Personalized Ranking loss	Standard method for implicit feedback

2.2. Main Models (Deep Learning)

(a) LightGCN (He et al., SIGIR 2020) — *Graph-based CF*

- **Input representation:** A bipartite user–game graph; each node has an initial embedding $e^{(0)}$.

- **Propagation:** $e_u^{(k+1)} = \sum_{i \in N_u} \frac{1}{\sqrt{|N_u||N_i|}} e_i^{(k)}$ (and symmetrically for items), without feature transformation or non-linearity.
- **Final embedding:** Weighted average over layers; score $\hat{y}_{ui} = e_u^\top e_i$.
- **Training:** BPR loss with negative sampling.
- **Motivation:** Exploits high-order connectivity in the graph (directly related to the *Link Analysis* part of the course) and mitigates sparsity.

(b) SASRec (Kang & McAuley, ICDM 2018) — *Sequential recommendation with Transformers*

- **Input representation:** The user's n most recent games (in chronological order), item embeddings + positional embeddings.
- **Model:** Stacked unidirectional (causal) self-attention blocks that predict the next game.
- **Training:** Binary cross-entropy with one negative sample per position.
- **Prediction:** Take the last hidden state and compute its dot product with all item embeddings.
- **Motivation:** Captures how preferences evolve over time; SASRec was originally evaluated on the Steam dataset, so reference results are available.

2.3. Proposed Improvement: Content-aware Hybrid

To address **cold start** and the **long tail**, we propose incorporating game content information into the CF models:

1. **Text encoding:** Use Sentence-BERT (`all-MiniLM-L6-v2`) to encode `title + tags + description` of each game into a vector $c_i \in \mathbb{R}^{384}$.
2. **Fusion:** The initial item embedding in LightGCN/SASRec is computed as $e_i^{(0)} = W c_i + \text{id_emb}_i$, where W is a learnable projection matrix. For new games (with no learned ID embedding), the model can still use $W c_i$.
3. **Behavior-based weighting (optional):** Use playtime and review labels (a simple form of Opinion Mining) to assign a confidence to each interaction, e.g., $w_{ui} = 1 + \alpha \log(1 + \text{playtime}_{ui})$, and down-weight interactions with a "Not Recommended" review.

Hypothesis to test: The hybrid model significantly improves Recall/NDCG on games with few interactions (cold/long-tail items) without hurting overall performance.

3. Benchmark Datasets

Both datasets are public and widely used in recommender system research:

Dataset	Source	Scale	Content
Steam Video Game and Bundle Data (McAuley Lab, UCSD)	https://cseweb.ucsd.edu/~jmcauley/datasets.html#steam_data	~7.8M reviews; ~88K users, ~10K games, ~5M ownership records (Australian users)	Reviews (with posting date), owned games + playtime (no timestamps), game metadata. The review data is the benchmark used in the SASRec paper.
Game Recommendations on Steam (Kaggle, A. Kozyriev, 2023)	https://www.kaggle.com/datasets/antonkozyriev/game-recommendations-on-steam	~41M reviews, ~14M users, ~50K games	`recommendations.csv` (user, game, is_recommended, hours, date), `games.csv`, `games_metadata.json` (description, tags)

Preprocessing:

- Apply **k-core** filtering (keep users and games with ≥ 5 interactions) to reduce noise and size.
- For the Kaggle dataset, subsample (e.g., ~1M users) to fit Colab resources.
- **Interaction definition:** In the UCSD dataset, ownership data has no timestamps, so each **review** (with its posting date) is treated as an interaction, following the SASRec paper. Playtime from the ownership table is joined by (user, game) and used only as an interaction weight. The Kaggle dataset already provides `date`, `hours`, and `is_recommended` for every interaction.
- Sort interactions chronologically for the sequential model.
- **Data split:** Temporal leave-one-out (last interaction → test, second-to-last → validation, the rest → train), consistent across all models.
- **Cold-start set:** Hold out ~10% of games from the training set to evaluate recommendation of new games.

4. Implementation

- **Libraries:**
  - Data processing: `pandas`, `numpy`, `scipy.sparse`
  - Models: `PyTorch`, `RecBole` (a framework that already provides BPR, ItemKNN, LightGCN, SASRec — used as the foundation and to ensure reproducibility)
  - Content features: `sentence-transformers`
  - Visualization: `matplotlib`, `seaborn`
- **Our own implementation/modifications (beyond existing code):**
  1. A preprocessing pipeline that converts both Steam datasets into RecBole's atomic file format.
  2. A content-aware item embedding module (Sentence-BERT + projection) integrated into LightGCN and SASRec.
  3. An interaction weighting mechanism based on playtime/reviews in the loss function.
  4. Dedicated evaluation scripts for cold-start and long-tail settings, plus beyond-accuracy metrics.

- **Reproducibility:** All experiments use fixed random seeds and shared configuration files; code and notebooks are published on GitHub so results can be re-run on Colab.

5. Evaluation

5.1. Metrics

Metric	Meaning	Why we use it
Recall@K (K = 10, 20)	Fraction of relevant games appearing in the top-K	Measures how well we "hit" the games users actually play
NDCG@K	Takes the position of the relevant game in the list into account	Evaluates ranking quality — correct recommendations at the top matter more
MRR	Reciprocal rank of the first relevant game	Complementary ranking perspective
Catalog Coverage	Fraction of games appearing in at least one recommendation list	Measures diversity and reveals popularity bias
Novelty / Long-tail ratio	Fraction of less popular games among recommendations	Evaluates support for discovering indie games

- Use **full ranking** over all items (no sampled metrics), following Krichene & Rendle (KDD 2020), for reliable results.
- Report results separately on three groups: all items, long-tail items, and cold-start items.

5.2. Experimental Design

1. Compare all models (MostPop, ItemKNN, BPR-MF, LightGCN, SASRec, Hybrid) on both datasets.
2. **Ablation study:** hybrid with/without content features; with/without playtime weighting.
3. **Hyperparameter analysis:** number of LightGCN layers, SASRec sequence length and number of attention heads, embedding dimension.
4. **Qualitative analysis:** Case studies of a few users — comparing recommendations across models.
5. Discuss strengths and weaknesses of each method (accuracy, training time, interpretability, cold-start handling).

5.3. Expected Results

- Deep Learning models (LightGCN, SASRec) outperform the baselines on Recall/NDCG.
- SASRec has an advantage on datasets with clear temporal information.
- The hybrid model clearly improves cold-start performance and increases coverage.

6. Timeline

Tasks	Deliverables
Data collection, exploratory data analysis (EDA), preprocessing	EDA notebook, cleaned data
Implement and run baselines (MostPop, ItemKNN, BPR-MF)	Baseline results
Train LightGCN and SASRec	Main model results
Implement the content-aware hybrid module and playtime weighting	Proposed model
Ablation study, hyperparameter analysis, result discussion	Result tables, figures
Finalize LaTeX report (Overleaf), slides, presentation	Report (≤ 30 pages), slides, GitHub code

7. Risks and Mitigation

Risk	Mitigation
Data too large for Colab	Stricter k-core filtering, user subsampling; use sparse matrices
Limited GPU training time	Save checkpoints, reduce embedding dimension, prioritize the UCSD dataset (smaller)
Missing metadata for some games	Use tags/genres instead of descriptions; for games with no metadata, use only the ID embedding
Hybrid does not improve results	Still report ablation results and analyze the causes as a meaningful conclusion